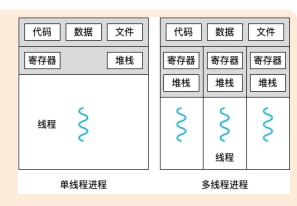
3. Java并发篇

- 多线程
  - Java 底层调用 pthread_create 创建线程，本质上 java 程序创建的线程，是和操作系统线程是一样的，是 1 对 1 的线程模型。
    - 多线程安全
      - 特性
        - 原子性 — 提供互斥访问，同一时刻只能有一个线程对数据进行操作，在Java中使用了atomic包和synchronized关键字来确保原子性
        - 可见性 — 一个线程对主内存的修改可以及时地被其他线程看到，在Java中使用了synchronized和volatile这两个关键字确保可见性
        - 有序性 — 一个线程观察其他线程中的指令执行顺序，由于指令重排序，该观察结果一般杂乱无序，在Java中使用了happens-before原则来确保有序性
      - 方法
        - synchronized关键字 排他性的同步机制 — 使用synchronized关键字来同步代码块或方法，确保同一时刻只有一个线程可以访问这些代码。对象锁是通过synchronized关键字锁定对象的监视器（monitor）来实现的。 — `public synchronized void someMethod() { /* ... */ }`
        - ② volatile关键字
          - 保证可见性 — 当一个变量被 volatile 修饰时，线程对该变量的修改会立即刷新到主内存，其他线程读取该变量时会直接从主内存中获取最新值，而不会使用缓存，确保线程间的可见性。
          - 禁止指令重排序
            - 可以防止 JVM 和 CPU 对其修饰的变量进行指令重排序优化，确保变量的赋值操作在其后的操作之前完成，保证代码执行的顺序性。
              - 写-写（Write-Write）屏障 — 写 volatile 变量 前的操作，不能被重排序到 volatile 写之后。
              - 读-写（Read-Write）屏障 — 读 volatile 变量 后的操作，不能被重排序到 volatile 读之前。
              - 写-读（Write-Read）屏障
                - 写 volatile 变量 前的操作，不能被重排序到 volatile 读之后。
                - 读 volatile 变量 后的操作，不能被重排序到 volatile 写之前。
            - 重排序
              - 原理 — 编译器 和 CPU 在执行程序时，为了提高执行效率和性能，按照特定的规则对指令执行顺序进行调整。
              - 条件
                - 在单线程环境下不能改变程序运行的结果
                - 存在数据依赖关系的不允许重排序
          - `private static volatile Singleton instance;`
          - 不能保证原子性，因此不能完全保证线程安全
        - Lock接口和ReentrantLock类 — java.util.concurrent.locks.Lock接口提供了比synchronized更强大的锁定机制，ReentrantLock是一个实现该接口的例子，提供了更灵活的锁管理和更高的性能。 — `private final ReentrantLock lock = new ReentrantLock(); lock.lock(); lock.unlock();`
        - 原子类 — Java并发库（java.util.concurrent.atomic）提供了原子类，如AtomicInteger、AtomicLong等，这些类提供了原子操作，可以用于更新基本类型的变量而无需额外的同步。原理是CAS — `AtomicInteger counter = new AtomicInteger(0); counter.incrementAndGet();`
        - 线程局部变量 — ThreadLocal类可以为每个线程提供独立的变量副本，这样每个线程都拥有自己的变量，消除了竞争条件。 — `ThreadLocal<Integer> threadLocalVar = new ThreadLocal<>(); threadLocalVar.set(10);`
        - 并发集合 — 使用java.util.concurrent包中的线程安全集合，如ConcurrentHashMap、ConcurrentLinkedQueue等，这些集合内部已经实现了线程安全的逻辑。
        - JUC工具类 — 使用java.util.concurrent包中的一些工具类可以用于控制线程间的同步和协作。例如：Semaphore和CyclicBarrier等。

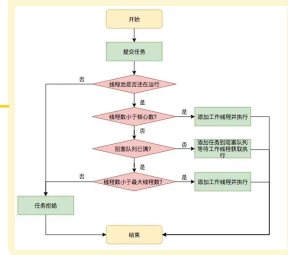
- 单例模式
  - 为什么使用 volatile?
    - 防止指令重排序，避免返回未完全初始化的对象。
    - 保证可见性，确保线程间及时看到最新的对象。
  - 为什么使用 synchronized?
    - 保证原子性，防止多个线程同时创建对象。
    - 实现互斥，确保只有一个线程执行初始化代码。
  - 如果去掉 volatile，会怎样? — 可能出现指令重排序，导致其他线程获取到未完全初始化的对象，出现程序异常。
  - 如果去掉 synchronized，会怎样? — 多个线程可能同时执行创建逻辑，导致多次创建单例对象，破坏单例模式。

- ① 线程
  - ① 状态
    - NEW — 尚未启动的线程状态，即线程创建，还未调用start方法
    - RUNNABLE — 就绪状态（调用start，等待调度）+正在运行
    - BLOCKED — 等待监视器锁时，陷入阻塞状态
      - 触发条件 — 线程进入BLOCKED状态通常是因为试图获取一个对象的锁（monitor lock），但该锁已经被另一个线程持有。这通常发生在尝试进入synchronized块或方法时，如果锁已被占用，则线程将被阻塞直到锁可用。
      - 唤醒机制 — 当一个线程被阻塞等待锁时，一旦锁被释放，线程将有机会重新尝试获取锁。如果锁此时未被其他线程获取，那么此线程可以从BLOCKED状态变为RUNNABLE状态。
    - WAITING — 等待状态的线程正在等待另一线程执行特定的操作（如notify）
      - 触发条件 — 线程进入WAITING状态是因为它正在等待另一个线程执行某些操作，例如调用Object.wait()方法、Thread.join()方法或LockSupport.park()方法。在这种状态下，线程将不会消耗CPU资源，并且不会参与锁的竞争。
      - 唤醒机制 — 线程在WAITING状态中需要被显式唤醒。例如，如果线程调用了Object.wait()，那么它必须等待另一个线程调用同一对象上的Object.notify()或Object.notifyAll()方法才能被唤醒。如果线程没有获取到锁则会进入 Waiting 状态，本质上执行了 LockSupport.park() 方法，解锁的时候会执行 LockSupport.unpark(Thread)，与上面park方法对应，给出许可证，解除等待状态。
        - notify 和 notifyAll 的区别
          - 醒等待的线程，同样最多只有一个线程能获得锁，同样不能控制哪个线程获得锁。
            - notify — 唤醒一个线程，其他线程依然处于wait的等待唤醒状态，如果被唤醒的线程结束时没调用notify，其他线程就永远没人去唤醒，只能等待超时，或者被中断
            - notifyAll — 所有线程退出wait的状态，开始竞争锁，但只有一个线程能抢到，这个线程执行完后，其他线程又会有一个幸运儿脱颖而出得到锁
          - notify选择唤醒的线程是任意的，但是依赖于具体实现的jvm。 — JVM有很多实现，比较流行的就是hotspot，hotspot对notofy()的实现并不是我们以为的随机唤醒，而是"先进先出"的顺序唤醒。
    - TIMED_WAITING — 具有指定等待时间的等待状态
    - TERMINATED — 线程完成执行，终止状态
  - 创建方式
    - 继承Thread类 — 用户自定义类继承java.lang.Thread类，重写其run()方法，run()方法中定义了线程执行的具体任务
      - 优点：如果需要访问当前线程，无需使用Thread.currentThread()方法，直接使用this，即可获得当前线程
      - 缺点：因为线程类已经继承了Thread类，所以不能再继承其他的父类
    - 实现Runnable接口 — 实现Runnable接口需要重写run()方法，然后将此Runnable对象作为参数传递给Thread类的构造器，创建Thread对象后调用其start()方法启动线程。
      - 优点：线程类只是实现了Runable接口，还可以继承其他的类
      - 缺点：如果需要访问当前线程，必须使用Thread.currentThread()方法
    - 实现Callable接口与FutureTask — 类似于Runnable，但Callable的call()方法可以有返回值并且可以抛出异常。要执行Callable任务，需将它包装进一个FutureTask，因为Thread类的构造器只接受Runnable参数，而FutureTask实现了Runnable接口。
      - 优点：线程只是实现Runnable或实现Callable接口，还可以继承其他类。
      - 缺点：如果需要访问当前线程，必须使用Thread.currentThread()方法
    - 使用线程池（Executor框架） — 提供了线程池的支持，这是一种更高效的线程管理方式，避免了频繁创建和销毁线程的开销。可以通过Executors类的静态方法创建不同类型的线程池。
      - 优点：线程池可以重用预先创建的线程，避免了线程创建和销毁的开销，显著提高了程序的性能。对于需要快速响应的并发请求，线程池可以迅速提供线程来处理任务，减少等待时间。并且，线程池能够有效控制运行的线程数量，防止因创建过多线程导致的系统资源耗尽（如内存溢出）。通过合理配置线程池大小，可以最大化CPU利用率和系统吞吐量。
      - 缺点：程池增加了程序的复杂度，特别是当涉及到线程池参数调整和故障排查时。错误的配置可能导致死锁、资源耗尽等问题，这些问题的诊断和修复可能较为复杂。
  - ② 停止线程运行
    - stop()暴力停止 — 线程调用stop()方法会被暴力停止，方法已弃用，该方法会有不好的后果：强制让线程停止有可能使一些请理性的工作得不到完成。
    - Interrupt
      - 每个线程都有一个与之关联的布尔属性来表示其中断状态，中断状态的初始值为false，当一个线程被其它线程调用Thread.interrupt()方法中断时，会根据实际情况做出响应。
        - 如果该线程正在执行低级别的可中断方法（如Thread.sleep()、Thread.join()或Object.wait()），则会解除阻塞并抛出InterruptedException异常。
        - 否则Thread.interrupt()仅设置线程的中断状态，在该被中断的线程中稍后可通过轮询中断状态来决定是否要停止当前正在执行的任务。
      - 异常法停止 — 线程调用interrupt()方法后，在线程的run方法中判断当前对象的interrupted()状态，如果是中断状态则抛出异常，达到中断线程的效果。
      - 使用return停止线程 — 调用interrupt标记为中断状态后，在run方法中判断当前线程状态，如果为中断状态则return，能达到停止线程的效果。
      - 在沉睡中停止 — 先将线程sleep，然后调用interrupt标记中断状态，interrupt会将阻塞状态的线程中断。会抛出中断异常，达到停止线程的效果。
  - ① **进程、线程、协程区别**



    - 进程
      - 概念 — 进程是程序在**运行时的实例**，是**操作系统分配资源的基本单位**
      - 特点
        - **进程之间独立**，互不影响
        - 进程拥有**独立的内存空间** — 不能直接访问其他进程的数据
        - 进程创建、销毁成本较高，调度相对线程更慢
        - 每个进程都有**独立的内存空间、文件描述符、系统资源**等。
    - 线程
      - 概念 — 线程是 **CPU调度和分派的基本单位**，属于**进程中的一个执行单元**
      - 特点
        - 多个线程**共享同一进程的内存空间**
          - 线程是轻量级的
          - **线程共享的**
            - 进程的堆
            - 方法区
            - 文件句柄、数据库连接
          - **线程不共享的**
            - 线程的栈
            - CPU 寄存器 — 程序计数器
            - ThreadLocal
        - **由操作系统调度**
        - 线程之间通信较快
    - 协程
      - 概念 — 协程是一种**比线程更轻量**的并发实现，**属于用户态的调度**
      - 特点
        - 由**用户代码管理调度**，而不是**由操作系统调度** — 减少了上下文切换的开销
        - **共享进程内存**
          - **协程共享的**
            - 进程的堆
            - 文件、数据库连接
            - 网络连接
            - 方法区
          - **协程不共享的**
            - 栈
            - 寄存器
            - ThreadLocal
        - 协程可以挂起和恢复，提高资源利用率

- ① 线程池
  - 原理

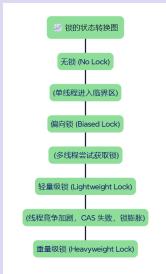


    - 核心线程池，线程池的最大容量，还有等待任务的队列
  - 参数
    - corePoolSize 线程池核心线程数量 — 默认情况下，线程池中线程的数量如果 <= corePoolSize，那么即使这些线程处于空闲状态，那也不会被销毁。
      - CPU密集型 — corePoolSize = CPU核数 + 1（避免过多线程竞争CPU）
      - IO密集型 — corePoolSize = CPU核数 x 2（或更高，具体看IO等待时间）
    - maximumPoolSize 线程池中最多可容纳的线程数量
      - ① 新任务交给线程池
        - 如果线程池中有空闲的线程，就会直接执行
        - 如果没有空闲的线程且当前线程池的线程数量小于corePoolSize 核心线程数量 — 就会创建新的线程来执行任务，否则就会将该任务加入到阻塞队列中 — 如果阻塞队列满了，就会创建一个新线程，从阻塞队列头部取出一个任务来执行，并将新任务加入到阻塞队列末尾。
        - 如果当前线程池中线程的数量等于maximumPoolSize，就不会创建新线程，就会去执行拒绝策略。
    - keepAliveTime — 线程池中线程的数量大于corePoolSize 核心线程数，并且某个线程的空闲时间超过了keepAliveTime，那么这个线程就会被销毁。
    - unit — keepAliveTime时间的单位
    - workQueue 工作队列 — 没有空闲的线程执行新任务时，该任务就会被放入工作队列中，等待执行。
    - threadFactory 线程工厂 — 可以用来给线程取名字等
    - handler 拒绝策略
      - CallerRunsPolicy — 使用线程池的调用者所在的线程去执行被拒绝的任务，除非线程池被停止或者线程池的任务队列已有空缺
      - AbortPolicy — 直接抛出一个任务被线程池拒绝的异常
      - DiscardPolicy — 不做任何处理，静默拒绝提交的任务
      - DiscardOldestPolicy — 抛弃最老的任务，然后执行该任务
      - RejectedExecutionHandler接口自定义拒绝策略
  - 种类
    - ScheduledThreadPool — 可以设置定期的执行任务 — 支持定时或周期性执行任务，比如每隔 10 秒钟执行一次任务，通过这个实现类设置定期执行任务的策略。
    - FixedThreadPool — 核心线程数和最大线程数是一样的， — 可以把它看作是固定线程数的线程池，它的特点是线程池中的线程数除了初始阶段需要从 0 开始增加外，之后的线程数量就是固定的，就算任务数超过线程数，线程池也不会再创建更多的线程来处理任务，而是会把超出线程处理能力的任务放到任务队列中进行等待。而且就算任务队列满了，到了本该继续增加线程数的时候，由于它的最大线程数和核心线程数是一样的，所以也无法再增加新的线程了。
    - CachedThreadPool — 可缓存线程池 — 特点在于线程数是几乎可以无限增加的（实际最大可以达到 Integer.MAX_VALUE，为 $2^{31}-1$），而当线程闲置时还可以对线程进行回收。也就是说该线程池的线程数量不是固定不变的，当然它也有一个用于存储提交任务的队列，但这个队列是 SynchronousQueue，队列的容量为0，实际不存储任何任务，它只负责对任务进行中转和传递，所以效率比较高。
    - SingleThreadExecutor — 使用唯一的线程去执行任务 — 原理和 FixedThreadPool 是一样的，只不过这里线程只有一个，如果线程在执行任务的过程中发生异常，线程池也会重新创建一个线程来执行后续的任务。这种线程池由于只有一个线程，所以非常适合用于所有任务都需要按被提交的顺序依次执行的场景，而前几种线程池不一定能够保障任务的执行顺序等于被提交的顺序，因为它们是多线程并行执行的。
    - SingleThreadScheduledExecutor — 实际和 ScheduledThreadPool 线程池非常相似，它只是 ScheduledThreadPool 的一个特例，内部只有一个线程。
  - ① 问题 — 禁止使用Executors 类定中一些快捷的工具方法来创建线程池，应该手动 new ThreadPoolExecutor 来创建线程池
    - 需要根据自己的场景、并发情况来评估线程池的几个核心参数
    - 任何时候，都应该为自定义线程池指定有意义的名称，以方便排查问题
  - ② shutdown() & shutdownNow()
    - shutdown — 使用了以后会置状态为SHUTDOWN，正在执行的任务会继续执行下去，没有被执行的则中断。此时，则不能再往线程池中添加任何任务，否则将会抛出 RejectedExecutionException 异常
    - shutdownNow — 使用了以后会置状态为STOP，并试图停止所有正在执行的线程，不再处理还在池队列中等待的任务，当然，它会返回那些未执行的任务。 — 通过调用 Thread.interrupt() 方法来实现的，但是这种方法的作用有限，如果线程中没有sleep 、wait、Condition、定时锁等应用, interrupt()方法是无法中断当前的线程的。所以，ShutdownNow()并不代表线程池就一定立即就能退出，它可能必须要等待所有正在执行的任务都执行完成了才能退出。
  - Future — 当向线程池提交任务时，会得到一个Future对象，它提供了几种方法来管理任务的执行，包括取消任务。
    - 取消任务 — Future接口中的cancel(boolean mayInterruptIfRunning)方法 — 参数mayInterruptIfRunning指示是否允许中断正在执行的任务。如果设置为true，则表示如果任务已经开始执行，那么允许中断任务；如果设置为false，任务已经开始执行则不会被中断。

- java.util.concurrent
  - 线程池
    - ThreadPoolExecutor — 最核心的线程池类，用于创建和管理线程池。通过它可以灵活地配置线程池的参数，如核心线程数、最大线程数、任务队列等，以满足不同的并发处理需求。
    - Executors — 线程池工厂类，提供了一系列静态方法用来创建不同类型的线程池，如newFixedThreadPool（创建固定线程数的线程池）、newCachedThreadPool（创建可缓存的线程池）、newSingleThreadExecutor（创建单线程线程池）等，方便开发者快速创建线程池。
  - 同步工具
    - CountDownLatch — 让一个或多个线程等待其他线程完成某些操作。它使用一个计数器（count）来控制线程的等待与唤醒。多个线程并发执行任务，主线程需要等待它们都执行完毕后再继续后续操作。需要初始化计数器。不可重用。
    - CyclicBarrier — 允许一组线程互相等待，直到所有线程都到达一个指定的屏障点，然后它们可以继续执行后续操作。CyclicBarrier 的一个重要特点是它是可重用的，即在一轮线程协作完成后，屏障可以被重置并再次使用。需要初始化计数器。
    - Semaphore — 通过计数信号量来管理线程访问资源的数量，能够有效地控制并发量。每个 Semaphore 维护一个许可计数，线程通过 acquire() 方法获取许可，使用 release() 方法释放许可。如果没有可用的许可，线程会被阻塞，直到有许可被释放。
  - 原子类
    - AtomicInteger — 原子整数类，提供了对整数类型的原子操作，如自增、自减、比较并交换等。通过硬件级别的原子指令来保证操作的原子性和线程安全性，避免了使用锁带来的性能开销，在多线程环境下对整数进行计数、状态标记等操作非常方便。
    - AtomicReference — 原子引用类，用于对对象引用进行原子操作。可以保证在多线程环境下，对对象的更新操作是原子性的，即要么全部成功，要么全部失败，不会出现数据不一致的情况。常用于实现无锁数据结构或需要对对象进行原子更新的场景。
  - 并发集合
    - ConcurrentHashMap — 线程安全的哈希映射表(具体信息可在Java集合篇查看)
    - CopyOnWriteArrayList — 线程安全的列表(具体信息可在Java集合篇查看)

- 数据一致性方案
  - 事务管理 — 使用数据库事务来确保一组数据库操作要么全部成功提交，要么全部失败回滚。 — 通过ACID（原子性、一致性、隔离性、持久性）属性，数据库事务可以保证数据的一致性。
  - 锁机制 — 使用锁机制来实现对共享资源的互斥访问
    - synchronized
    - ReentrantLock
  - 版本控制 — 通过乐观锁的方式，在更新数据时记录数据的版本信息，从而避免同时对同一数据进行修改，进而保证数据的一致性。

- 常用锁
  - 读写锁（ReadWriteLock） — 一种读写分离的锁，允许多个线程同时读取，但写操作需要独占锁。 — 读操作远多于写操作的场景（如缓存、配置文件读取）。 — `private ReadWriteLock lock = new ReentrantReadWriteLock(); private Lock readLock = lock.readLock(); private Lock writeLock = lock.writeLock();`
  - 自旋锁（SpinLock） — 一种轻量级锁，线程在等待锁时不断循环尝试获取锁，而不是直接进入阻塞态。
    - 适用于锁持有时间短、线程切换成本高的场景。
    - 适合多核 CPU 环境，避免上下文切换。
    - 如果锁占用时间长，自旋会消耗大量 CPU。
  - StampedLock — 一种改进的读写锁，提供乐观读、悲观读、写锁，减少不必要的写阻塞。
    - 适用于读多写少，且需要较高性能的并发场景。
    - 适用于缓存读取、统计类操作。
  - ① 乐观锁和悲观锁
    - 乐观锁
      - 假设数据不会被修改，不加锁而是使用 CAS机制更新数据，失败时重试。适用于读多写少，竞争不激烈的场景，避免线程阻塞提高性能。 — AtomicInteger、AtomicLong
      - 实现方式
        - 版本号控制 — 增加一个版本号字段记录数据更新时候的版本，每次更新时递增版本号。在更新数据时，比较当前版本号和更新前获取的版本号是否一致，一致则更新成功，否则失败。
        - 时间戳 — 使用时间戳记录数据的更新时间，在更新数据时，在比较时间戳。如果当前时间戳大于数据的时间戳，则说明数据已经被其他线程更新，更新失败。
        - CAS操作
          - 使用 CAS 操作实现了线程安全的原子操作，可以用来实现乐观锁。基于循环重试的机制，如果CAS操作一直未能成功，线程会一直自旋重试，占用CPU资源。 — CAS 操作依赖于 CPU 提供的原子指令确保了对共享变量的更新操作是不可分割的
          - 缺点
            - 循环时间长开销大 — 自旋CAS的方式如果长时间不成功，会给CPU带来很大的开销。
            - 只能保证一个共享变量的原子操作 — 只对一个共享变量操作可以保证原子性，但是多个则不行，多个可以通过AtomicReference来处理或者使用锁synchronized实现。
            - ① ABA问题
              - 无法检测到值已经被修改过，即使最后它的值与初始值相同，但中间的变化可能导致错误的操作。
              - 例子
                - 1. 线程 1 读取内存值为 A
                - 2. 线程 2 修改该值：A → B → A
                - 3. 线程 1 执行 CAS，发现当前值仍是 A，操作成功，但此时实际已发生过中间修改
              - 解决方法
                - AtomicStampedReference — 通过维护一个整数版本号，确保即使值未变化，版本号变化也能检测到数据的中间更改
                - AtomicMarkableReference — 使用布尔标志位标识数据是否被更改 适用于不需要精确记录次数，只关心是否发生过变化的情况。
    - 悲观锁 — 假设数据会被修改，访问数据前直接加锁，确保数据一致性。适用于写操作多、竞争激烈、需要强一致性的场景。
      - 内置锁（synchronized）
        - 隐式锁，适用于对象方法、静态方法或代码块的加锁，确保同一时间只有一个线程能访问同步代码。
          - 锁的升级过程


锁的状态转换图
无锁 (No Lock)
(单线程进入临界区)
偏向锁 (Biased Lock)
(多线程尝试获取锁)
轻量级锁 (Lightweight Lock)
(线程竞争加剧，CAS 失败，锁膨胀)
重量级锁 (Heavyweight Lock)

            - 无锁（No Lock） — 只有一个线程访问临界区，且未发生竞争。
            - 偏向锁（Biased Locking） — 只有一个线程多次访问同一锁对象。
              - 第一次进入同步块时，锁会偏向当前线程，记录线程 ID 在 Mark Word 中。
              - 后续相同线程再次获取锁时，只需对比 线程 ID，若相同，直接进入同步块。
              - 锁撤销：当其他线程尝试获取该锁时，偏向锁会撤销，升级为轻量级锁。
            - 轻量级锁（Lightweight Lock） — 多个线程交替访问临界区，但没有发生竞争。
              - 线程尝试获取锁时，会在其虚拟机栈中创建一个 Lock Record，用于存储对象 Mark Word 的副本。
              - 然后使用 CAS 操作将对象头的 Mark Word 替换为 Lock Record 的指针。
              - 如果 CAS 操作成功，表示获取锁成功，进入同步代码块。
              - 解锁时，将对象头恢复原始 Mark Word，并使用 CAS 操作释放锁。
              - 如果 CAS 操作失败，表示存在其他线程竞争锁，升级为重量级锁。
            - 重量级锁（Heavyweight Lock）
              - 多个线程同时竞争同一个锁，发生严重冲突。
                - 如果轻量级锁 CAS 失败，锁会升级为重量级锁，通过 Monitor 对象实现互斥。
                - 线程无法通过自旋获取锁时，会被挂起，进入 BLOCKED 状态，等待锁释放。
                - 解锁时，会唤醒等待队列中的线程，竞争锁资源。
              - 缺点：涉及操作系统层面的上下文切换（用户态 ↔ 内核态），会带来严重的性能开销。
          - 使用方法
            - synchronized方法 — `public synchronized void increment() {}`
            - synchronized代码块 — `private Object lock = new Object(); public void increment() {synchronized (lock) {count++;}}`
        - 内置的并且使用者看不到的锁也被称为监视器锁
        - 原理
          - 依赖于 Monitor（监视器锁） 来实现同步，最终是通过字节码指令 monitorenter 和 monitorexit 来完成锁的获取和释放。
            - 执行流程
              - 1. monitorenter 指令（获取锁）
                - 会清空线程的本地缓存，强制从主内存中读取共享变量的最新值。
                - 如果目标对象的锁未被占用，当前线程会获取锁，并将锁的计数器 +1，同时将当前线程设置为持有该锁的线程。
                - 如果目标对象的锁已被其他线程占用，当前线程进入 EntryList（锁池）并进入阻塞状态，等待锁的释放。
              - 2. 执行同步代码块 — 获取锁的线程可以安全地访问共享资源，其他线程处于等待或阻塞状态，无法访问该资源。
              - 3. monitorexit 指令（释放锁）
                - 会刷新线程的本地缓存，将修改后的共享变量写回主内存，保证其他线程可以读取到最新数据。
                - 线程执行完同步代码块后，锁的计数器 -1，如果计数器归 0，表示锁被释放。
                - 处于 EntryList 中的线程会被唤醒并重新竞争锁，只有一个线程可以成功获取锁并继续执行。
          - 等待队列
            - EntryList（锁池） — 当多个线程尝试进入同步代码块但无法获取锁时，都会被加入到该队列中等待锁的释放。线程从该队列中被唤醒后重新一起竞争锁。
            - WaitSet（等待集） — 当线程调用 wait() 方法后，当前线程会释放锁，进入 WaitSet 等待被其他线程唤醒。线程被唤醒后会重新进入 EntryList 竞争锁。
        - 特点
          - 可重入性 — 底层是利用计算机系统mutex Lock实现的。每一个可重入锁都会关联一个线程ID和一个锁状态status。
            - 当一个线程请求方法时，会去检查锁状态。
              - 未被占用（count == 0） — 使用 CAS（Compare And Swap） 设置线程 ID，并将 count+1，获取锁成功。
              - 已被占用
                - 如果持有锁的线程与当前线程相同，说明是同一线程的重入，此时将 count+1，允许再次进入。
                - 如果是其他线程，则进入阻塞状态，放入等待队列。
            - 释放锁的过程
              - 将 Monitor 对象的重入计数器(count)-1
              - 当 count == 0，完全释放锁，唤醒等待队列中的线程尝试获取锁
        - JVM优化
          - 锁膨胀(锁升级)
            - JDK 1.6 之前，是重量级锁，在释放和获取锁时都会从用户态转换成内核态，转换效率是比较低的。
            - 有了锁膨胀机制之后，状态就多了无锁、偏向锁以及轻量级锁了，在进行并发操作时，大部分的场景都不需要用户态到内核态的转换了，大幅的提升了性能。
          - 锁消除 — 某些情况下，JVM如果检测不到某段代码被共享和竞争的可能性，就会将这段代码所属的同步锁消除掉，从而到底提高程序性能的目的。
          - 锁粗化 — 将多个连续的加锁、解锁操作连接在一起，扩展成一个范围更大的锁。
          - 自适应自旋锁 — 通过自身循环，尝试获取锁的一种方式。 — 避免一些线程的挂起和恢复操作，因为挂起线程和恢复线程都需要从用户态转入内核态，这个过程是比较慢的，通过自旋的方式可以一定程度上避免线程挂起和恢复所造成的性能开销。
      - 显式锁（ReentrantLock）
        - 显式锁，提供更灵活的锁机制，支持公平锁、非公平锁、可中断锁、尝试获取锁、超时等待等特性。
        - 原理 — 底层：依赖于 AbstractQueuedSynchronizer抽象类。AQS 是一个提供了基本同步机制的框架，其中包括了队列、状态值等。
          - AbstractQueuedSynchronizer — 是Java中的一个抽象类，一个用于构建锁、同步器、协作工具类的工具（框架）。
            - 通过一个状态值（state）和一个CLH（虚拟双向队列）来管理线程的获取和释放，支持独占模式和共享模式两种锁的实现。
              - state（同步状态）
                - 用于表示锁的状态，通常 0 表示未占用，大于 0 表示被线程占用。
                  - 根据具体实现类的不同而不同:
                    在Semapore里，他表示剩余许可证的数量;
                    在CountDownLatch里，它表示还需要倒数的数量;
                    在ReentrantLock中，state用来表示"锁"的占有情况，包括可重入计数，
                    当state的值为0的时候，标识该Lock不被任何线程所占有。
                  - state是volatile修饰的，并被并发修改，所以修改state的方法都需要保证线程安全
                - 通过 CAS（Compare and Swap） 操作修改，确保线程安全。
              - CLH（虚拟双向队列）FIFO
                - Node（队列节点）
                  - 等待线程的封装，用于维护线程、锁状态、前后指针，组成一个 CLH 双向队列。
                  - 节点状态
                    - SIGNAL：表示当前节点需要被唤醒。
                    - CANCELLED：表示超时或中断，节点已取消。
                    - CONDITION：节点等待在条件队列中，等待 condition.signal() 唤醒。
                    - PROPAGATE：共享锁模式下，状态需要传播。
                - head 和 tail（队列头尾指针） — 采用 CLH 队列 结构，线程通过 addWaiter() 方法加入 等待队列，保证公平性。
            - 两种锁模式
              - 独占模式（Exclusive） — 只有一个线程能获取锁，其他线程必须等待。 — 适用场景：ReentrantLock、ReentrantReadWriteLock 写锁。
              - 共享模式（Shared） — 多个线程可以同时获取锁，通常用于读多写少的场景。 — 适用场景：Semaphore、CountDownLatch、ReentrantReadWriteLock 读锁。
            - 加锁流程 — 需要每个实现类重写tryAcquire和tryRelease等方法。
              - 尝试获取锁（调用 tryAcquire() 方法）。
              - 获取失败，入队等待（addWaiter() 加入等待队列）。
              - 自旋等待锁（acquireQueued()，不断检查锁状态，直到获取锁）。
              - 获取锁成功，退出等待队列。
        - 特点
          - 公平锁和非公平锁
            - 公平锁 — 按照线程请求锁的顺序分配，避免线程饥饿，但性能略低。
              - 设置公平锁：`ReentrantLock fairLock = new ReentrantLock(true)`
              - 获取锁时，先将线程自己添加到等待队列的队尾并休眠，当某线程用完锁之后，会去唤醒等待队列中队首的线程尝试去获取锁，锁的使用顺序也就是队列中的先后顺序，在整个过程中，线程会从运行状态切换到休眠状态，再从休眠状态恢复成运行状态，但线程每次休眠和恢复都需要从用户态转换成内核态，而这个状态的转换是比较慢的，所以公平锁的执行速度会比较慢。
            - 非公平锁（默认） — 允许多个线程在同一时刻竞争锁，不考虑它们申请锁的顺序。 — 当线程获取锁时，会先通过 CAS 尝试获取锁，如果获取成功就直接拥有锁，如果获取锁失败才会进入等待队列，等待下次尝试获取锁。这样做的好处是，获取锁不用遵循先到先得的规则，从而避免了线程休眠和恢复的操作，这样就加速了程序的执行效率。
            - 使用方法 — `private Lock lock = new ReentrantLock(); lock.lock();lock.unlock();`
          - 可中断性 — 线程在等待锁的过程中，可以被其他线程中断而提前结束等待。在底层，ReentrantLock 使用了与 LockSupport.park() 和 LockSupport.unpark() 相关的机制来实现可中断性。
          - 设置超时时间 — ReentrantLock 支持在尝试获取锁时设置超时时间，即等待一定时间后如果还未获得锁，则放弃锁的获取。这是通过内部的 tryAcquireNanos 方法来实现的。
          - 多个条件变量 — 支持多个条件变量，每个条件变量可以与一个 ReentrantLock 关联。这使得线程可以更灵活地进行等待和唤醒操作，而不仅仅是基于对象监视器的 wait() 和 notify()。多个条件变量的实现依赖于 Condition 接口。 — `Condition condition = lock.newCondition(); condition.await(); condition.signal();`
          - 可重入性 — 支持可重入性，即同一个线程可以多次获得同一把锁，而不会造成死锁。这是通过内部的 holdCount 计数来实现的。当一个线程多次获取锁时，holdCount 递增，释放锁时递减，只有当 holdCount 为零时，其他线程才有机会获取锁。
      - 应用场景
        - Synchronized — 适用于简单同步需求和不需要额外锁功能的场景
        - ReentrantLock — 适用于需要更高级锁功能、性能优化或复杂同步逻辑的情况
      - ① 区别
        - 用法不同
          - synchronized 可用来修饰普通方法、静态方法和代码块
          - ReentrantLock 只能用在代码块上
        - 获取锁和释放锁方式不同
          - synchronized 会自动加锁和释放锁
          - ReentrantLock 需要手动加锁和释放锁
        - 锁类型不同
          - synchronized 属于非公平锁
          - ReentrantLock 既可以是公平锁也可以是非公平锁
        - 响应中断不同
          - synchronized 不能响应中断 — synchronized 是一种不可中断的阻塞锁，一旦线程进入等待状态，除非锁被释放，否则无法退出。
          - ReentrantLock 可以响应中断，解决死锁的问题 — 如果线程在等待锁时被 interrupt() 方法中断，锁的获取会立刻终止，抛出 InterruptedException，线程可以执行其他任务而不是永久等待。
        - 底层实现不同
          - synchronized 是 JVM 层面通过监视器实现的
          - ReentrantLock 是基于 AQS 实现的
  - ① 死锁
    - 条件(同时满足)
      - 互斥条件 — 多个线程不能同时使用同一个资源
      - 持有并等待条件 — 线程已经持有某些资源，同时还在等待其他线程占用的资源。
      - 不可剥夺条件 — 当线程已经持有了资源，在自己使用完之前不能被其他线程获取
      - 环路等待条件 — 在死锁发生的时候，两个线程获取资源的顺序构成了环形链
    - 解决方法 — 需要破坏其中一个条件就可以
      - 资源有序分配法 — 规定资源的申请顺序，所有线程按照统一顺序申请锁，避免循环等待 — 所有线程都按照相同的顺序获取锁
      - 使用 tryLock() 进行超时控制 — 避免线程无限等待锁
      - 避免嵌套锁

- Threadlocal
  - Java 提供的一种线程本地存储机制，主要用于在多线程环境下，每个线程都能维护自己的**独立副本** — 避免了多个线程对共享变量的并发访问问题，保证线程间的数据隔离。
  - 核心结构
    - 每个线程内部有一个 ThreadLocalMap，ThreadLocalMap 是 ThreadLocal 的静态内部类，类似于一个弱引用的 HashMap，用于存储 ThreadLocal 变量。 — ThreadLocal 对象对于每个线程是独立存储变量的，也就是说，创建了一个ThreadLocal对象可以在多个线程中使用，但是变量值是独立存在的。线程通过ThreadLocalMap存储每一个ThreadLocal对象。
    - 数据存储过程：
      - 每次调用 ThreadLocal 的 set() 方法时，会将数据存储到当前线程的 ThreadLocalMap 中。
      - ThreadLocalMap 以 ThreadLocal 对象为 key，用户设置的值为 value，实现线程间数据的隔离。
    - 优点
      - 线程隔离 — 为每个线程提供了独立的变量副本，线程之间不会相互影响，可以安全地在多线程环境中使用这些变量而不必担心数据竞争或同步问题。
      - 降低耦合度 — 在同一个线程内的多个函数或组件之间，使用ThreadLocal可以减少参数的传递，降低代码之间的耦合度，使代码更加清晰和模块化。
      - 性能优势 — 由于ThreadLocal避免了线程间的同步开销，所以在大量线程并发执行时，相比传统的锁机制，它可以提供更好的性能。
    - **内存泄漏** — 回收ThreadLocal实例后，ThreadLocalMap 中的 key 会变为 null，但对应的 value 仍然存在。 — 如果线程一直不结束（例如线程池中的线程），这些 key 为 null 的 Entry 会一直存在，导致 value 无法被回收，从而造成内存泄漏。
  - 原理
    - 当调用get()方法时，ThreadLocal会检查当前线程的ThreadLocalMap中是否有与之关联的值。
      - 如果有，返回该值；
      - 如果没有，会调用initialValue()方法（如果重写了的话）来初始化该值，然后将其放入ThreadLocalMap中并返回。
    - 当调用set()方法时，会将给定的值与当前线程关联起来，在当前线程的ThreadLocalMap中存储一个键值对，键是ThreadLocal对象自身，值是传入的值。
    - 当调用remove()方法时，会从当前线程的ThreadLocalMap中移除与该ThreadLocal对象关联的条目。

- 引用
  - 强引用（Strong Reference） — 强引用是 Java 中最常见的引用类型，它的生命周期由引用变量控制。也就是说，只要一个对象有强引用指向它，垃圾回收器就不会回收这个对象。 — `MyObject obj = new MyObject();` 在这个例子中，obj 是对 MyObject 的强引用。只要 obj 仍然存在，MyObject 对象就不会被垃圾回收。
  - 弱引用（Weak Reference）
    - 弱引用是 java.lang.ref.WeakReference 类提供的一种引用类型。
    - 如果一个对象只有弱引用指向它，那么当垃圾回收器运行时，只要该对象没有强引用，它就会被回收。换句话说，弱引用不会阻止对象被回收，即使这个对象没有任何强引用指向它。
      - `WeakReference<MyObject> weakRef = new WeakReference<>(new MyObject());`
      - 在这个例子中，weakRef 是 MyObject 的弱引用。如果没有其他强引用指向 MyObject 对象，垃圾回收器会在适当的时机回收 MyObject 对象，甚至即使 weakRef 仍然存在。

Presented with xmind